Task 02: Packet Sniffing, Firewall Configuration and Vulnerability Scanning

Submitted by: Khush Rupapara
Date: January 22, 2026
Course: VAPT(Intern)

Executive Summary

This report documents hands-on network security tasks including packet sniffing using Scapy, firewall configuration with iptables, and vulnerability scanning using OpenVAS. The objective was to gain practical experience with core security tools and techniques used by cybersecurity professionals.

All activities were performed on a Kali Linux system in a controlled environment. Key outcomes included capturing and analyzing 100 network packets to identify protocol distribution, implementing firewall rules to control network traffic, and conducting a vulnerability scan to identify and assess security weaknesses based on severity and impact.

Overall, the exercise strengthened practical skills in network traffic analysis, firewall management, vulnerability assessment, Python-based security tools, and Linux security configuration, closely reflecting real-world cybersecurity workflows.

1)Packet Sniffing with Scapy

What is Packet Sniffing?

Packet sniffing is the process of intercepting and logging network traffic that passes through a digital network. A packet sniffer captures data packets and decodes them to reveal the information contained within. Packet sniffers are essential tools for network troubleshooting, security analysis, intrusion detection, network performance monitoring, protocol analysis, and educational purposes.

Introduction to Scapy

Scapy is a powerful Python-based interactive packet manipulation program and library. It allows users to forge or decode packets of various protocols, send packets on the network, capture packets, perform scanning, tracerouting, probing, and network discovery.

Common Network Protocols

TCP (Transmission Control Protocol): A connection-oriented protocol that provides reliable, ordered delivery of data. Used for HTTP/HTTPS, email, and file transfers.

UDP (User Datagram Protocol): A connectionless protocol that sends datagrams without establishing a connection. Faster but less reliable than TCP. Used for DNS, streaming, and gaming.

ICMP (Internet Control Message Protocol): Used for diagnostic and control purposes in IP networks. Most commonly known for the ping command.

Installing Scapy and Dependencies

The first step was to install Scapy along with matplotlib for data visualization using pip:

$ pip install scapy matplotlib

```
┌──(venv)─(bobby㉿bobby)-[~/Desktop/Internship]
└─$ pip install scapy matplotlib
Collecting scapy
  Downloading scapy-2.7.0-py3-none-any.whl.metadata (5.8 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.8-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (52 kB)
Collecting contourpy ≥ 1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
Collecting cycler ≥ 0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools ≥ 4.22.0 (from matplotlib)
  Downloading fonttools-4.61.1-cp313-cp313-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl.metadata (114 kB)
Collecting kiwisolver ≥ 1.3.1 (from matplotlib)
  Downloading kiwisolver-1.4.9-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (6.3 kB)
Collecting numpy ≥ 1.23 (from matplotlib)
  Downloading numpy-2.4.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting packaging ≥ 20.0 (from matplotlib)
  Downloading packaging-25.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pillow ≥ 8 (from matplotlib)
  Downloading pillow-12.1.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
Collecting pyparsing ≥ 3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting python-dateutil ≥ 2.7 (from matplotlib)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting six ≥ 1.5 (from python-dateutil ≥ 2.7→matplotlib)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading scapy-2.7.0-py3-none-any.whl (2.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 7.1 MB/s  0:00:00
Downloading matplotlib-3.10.8-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (8.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.7/8.7 MB 9.6 MB/s  0:00:00
Downloading contourpy-1.3.3-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (362 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.61.1-cp313-cp313-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_5_x86_64.whl (4.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4.9/4.9 MB 7.7 MB/s  0:00:00
Downloading kiwisolver-1.4.9-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 8.1 MB/s  0:00:00
Downloading numpy-2.4.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.4/16.4 MB 8.3 MB/s  0:00:02
Downloading packaging-25.0-py3-none-any.whl (66 kB)
```

Generating Network Traffic for Testing

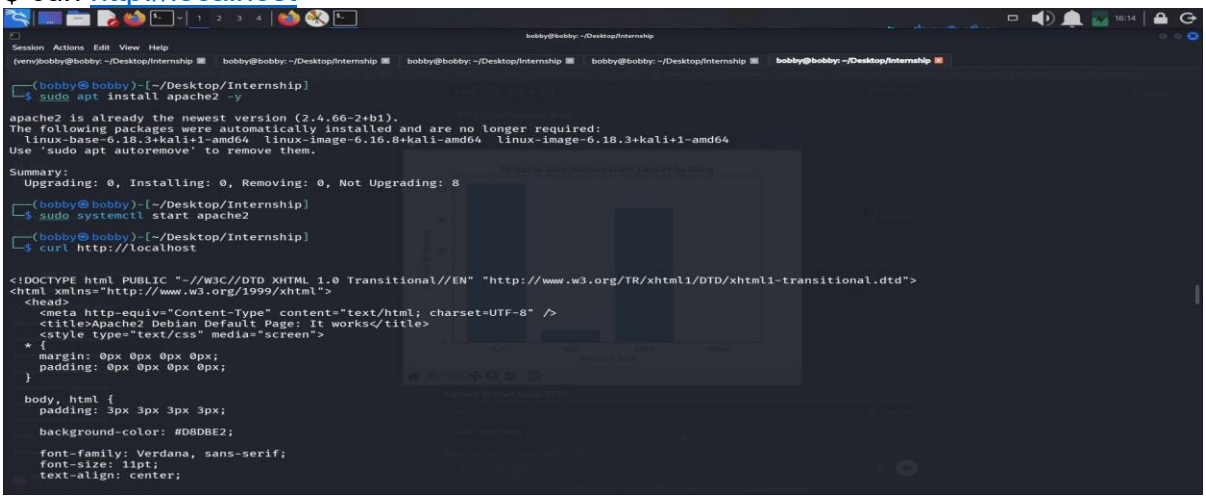
A. Apache2 Web Server Setup

An Apache2 web server was installed to generate HTTP traffic:

$ sudo apt install apache2 -y $ sudo systemctl start apache2

Testing the web server with curl:

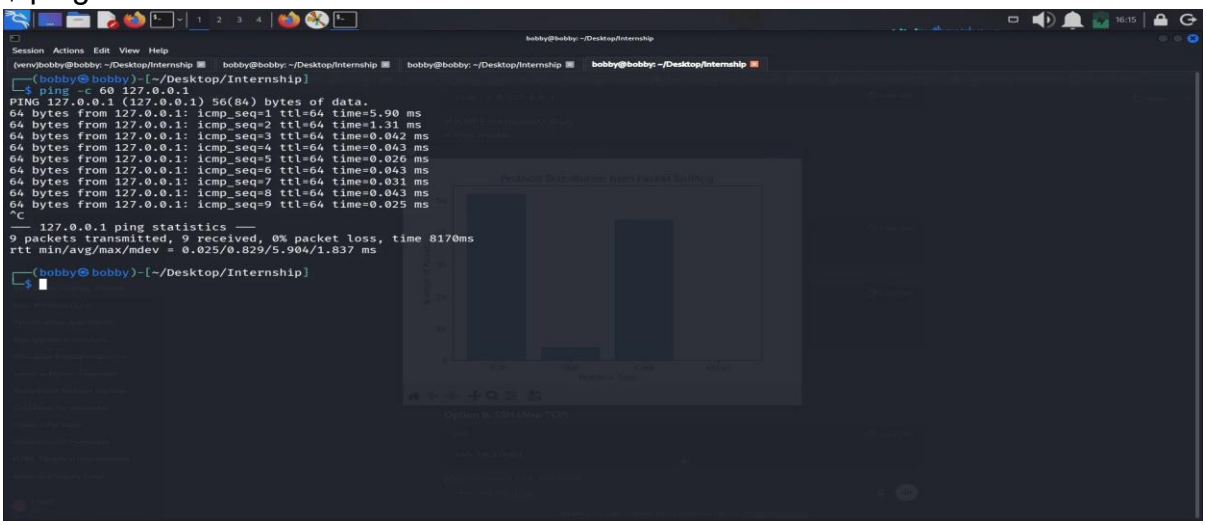
$ curl http://localhost



B. ICMP Traffic Generation

ICMP packets were generated using the ping command:

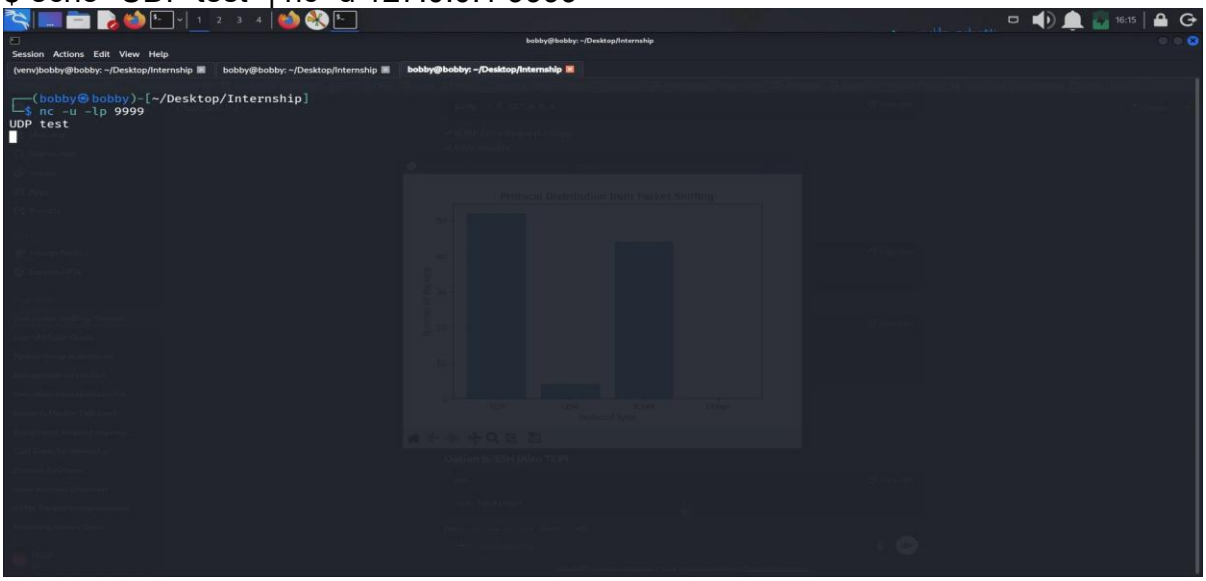
$ ping -c 60 127.0.0.1

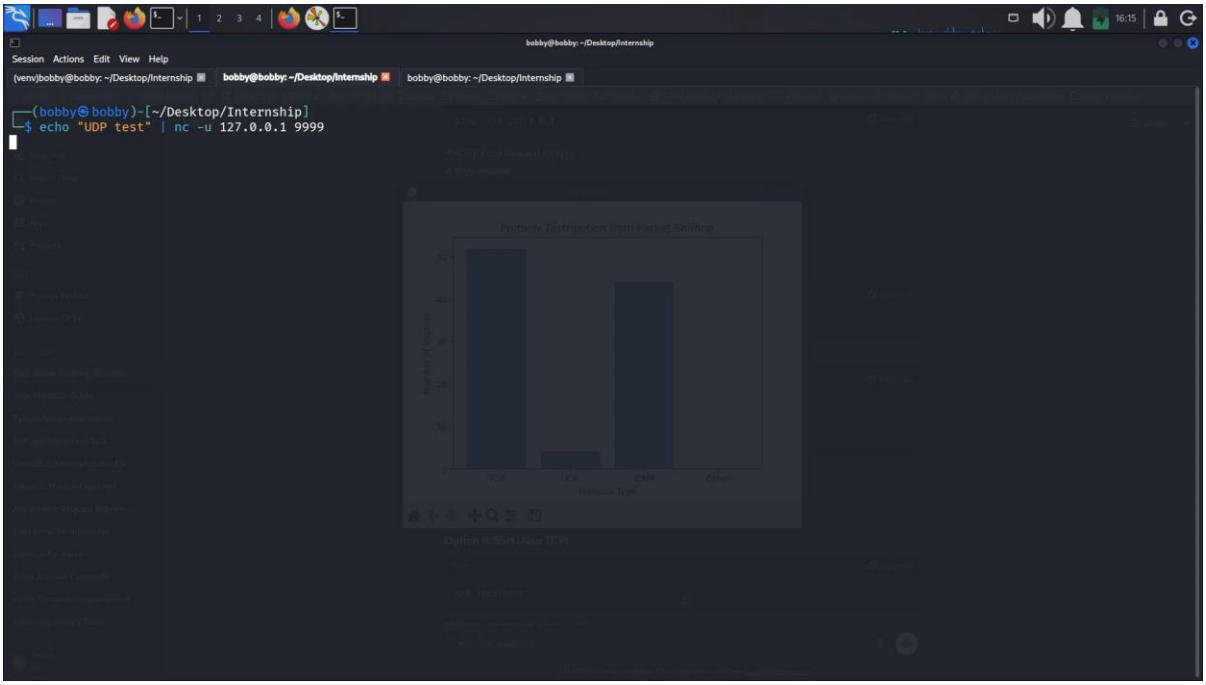


C. UDP Traffic Generation

UDP packets were generated using netcat:

$ echo "UDP test" | nc -u 127.0.0.1 9999

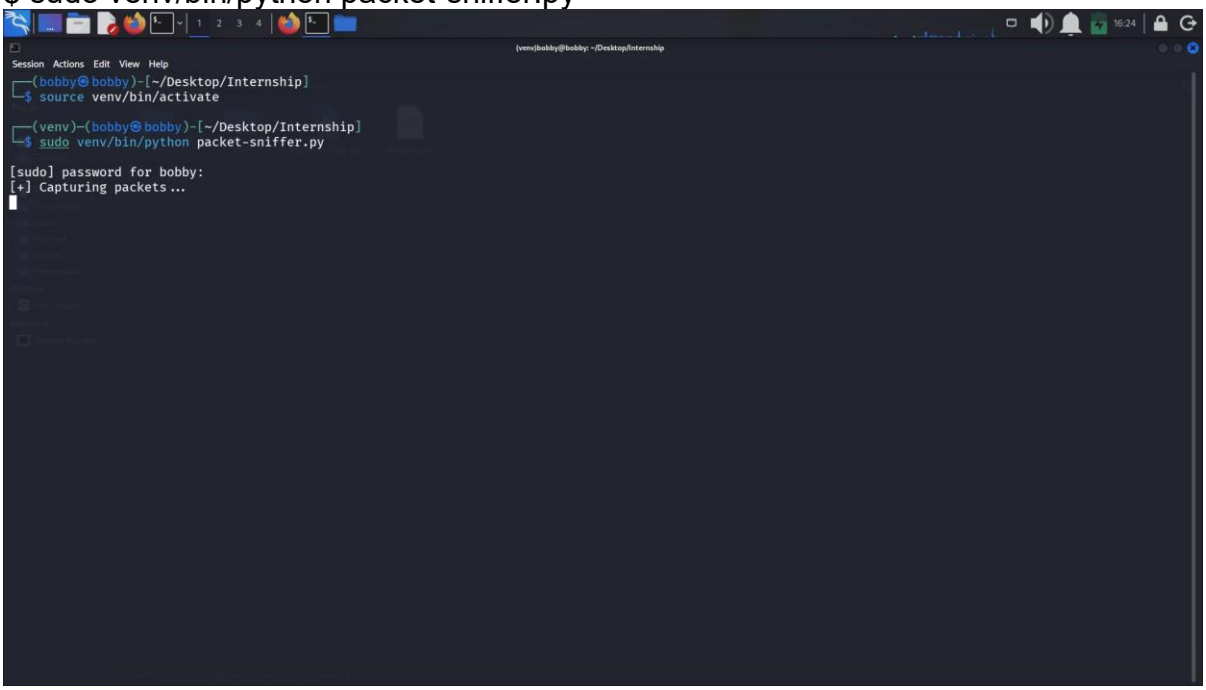




Executing the Packet Sniffer

The packet sniffer script was executed with sudo privileges:

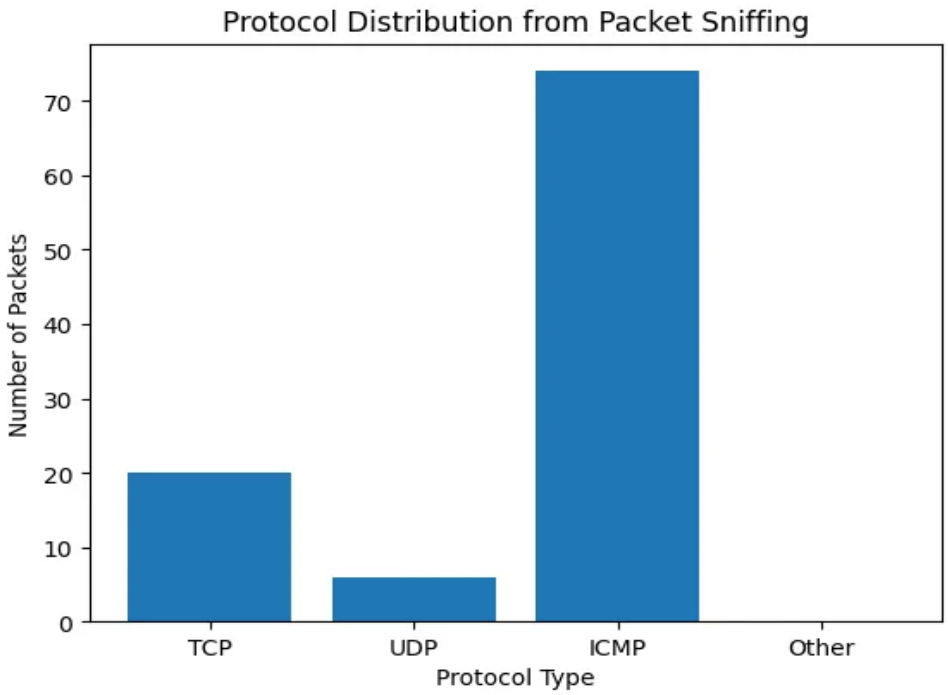
$ sudo venv/bin/python packet-sniffer.py



Results and Analysis

The packet analysis revealed the following protocol distribution from the 100 captured packets:

Protocol	Packet Count	Percentage
TCP	20	20%
UDP	6	6%
ICMP	74	74%
Total	**100**	**100%**



Analysis and Observations

ICMP Dominance (74%): The majority of captured traffic was ICMP, which aligns with extensive use of the ping command. Each ping generates both request and reply packets.

TCP Traffic (20%): TCP packets were generated by HTTP requests to Apache and the three-way handshake process for establishing connections.

UDP Traffic (6%): UDP packets were from netcat tests and background DNS queries.

2) Firewall Configuration with iptables

What is a Firewall?

A firewall is a network security system that monitors and controls incoming and outgoing network traffic based on predetermined security rules. Firewalls establish a barrier between trusted internal networks and untrusted external networks.

Introduction to iptables

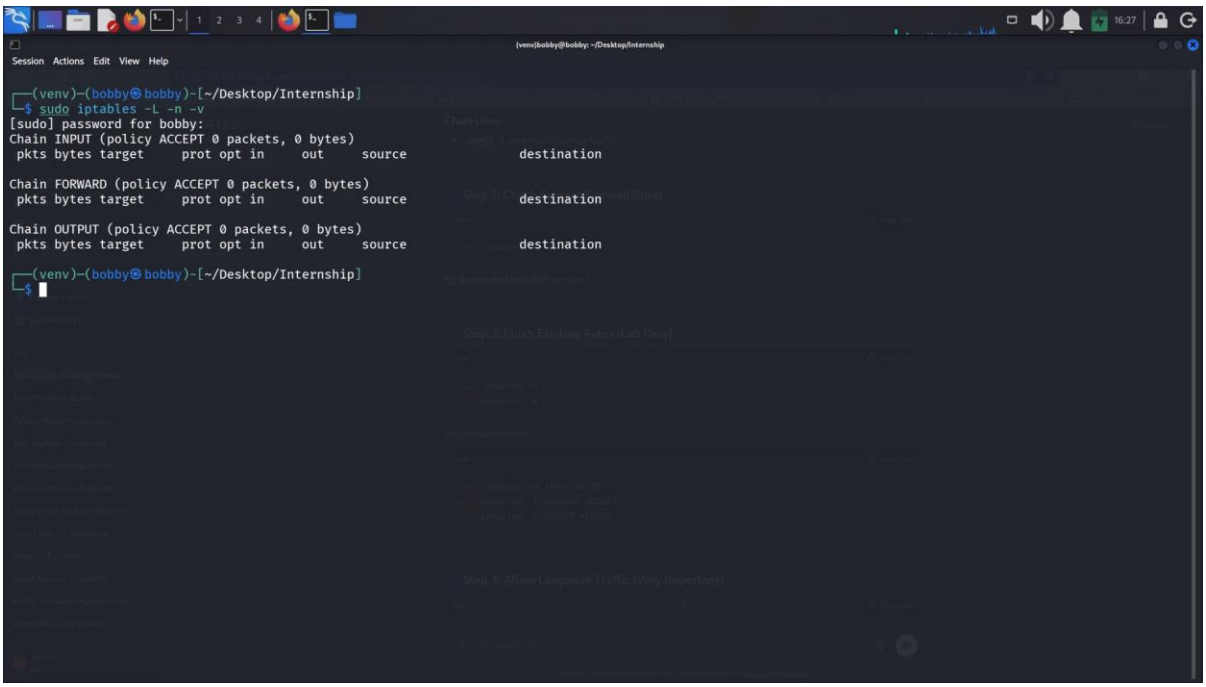
iptables is a user-space utility that allows system administrators to configure the Linux kernel firewall. It provides a powerful framework for packet filtering and network address translation (NAT).

Key concepts: Tables (filter, nat, mangle), Chains (INPUT, OUTPUT, FORWARD), Rules (matching criteria and actions), and Policies (default actions).

Checking Initial Firewall Status

The current iptables configuration was examined:

```
$ sudo iptables -L -n -v
```



Configuring Firewall Rules

Step 1: Allow Loopback Traffic

```
$ sudo iptables -A INPUT -i lo -j ACCEPT
```

Step 2: Allow Established and Related Connections

```
$ sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j
ACCEPT
```

Step 3: Allow SSH Traffic (Port 22)

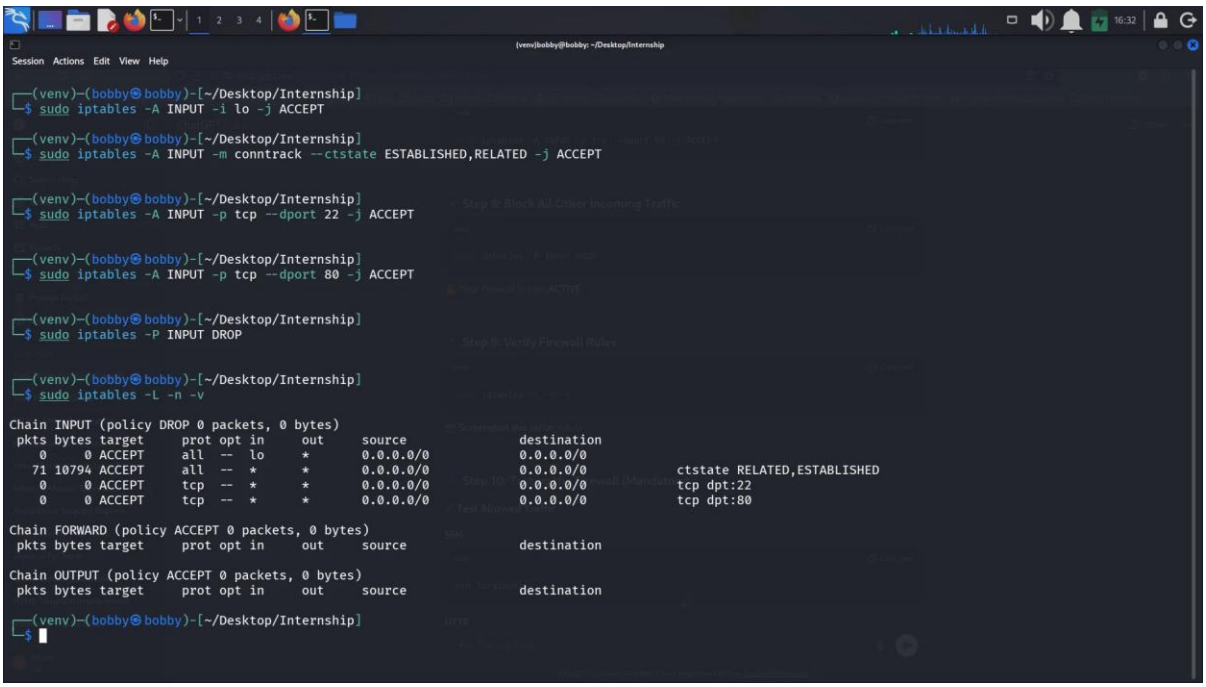
```
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

Step 4: Allow HTTP Traffic (Port 80)

```
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Step 5: Set Default DROP Policy

```
$ sudo iptables -P INPUT DROP
```



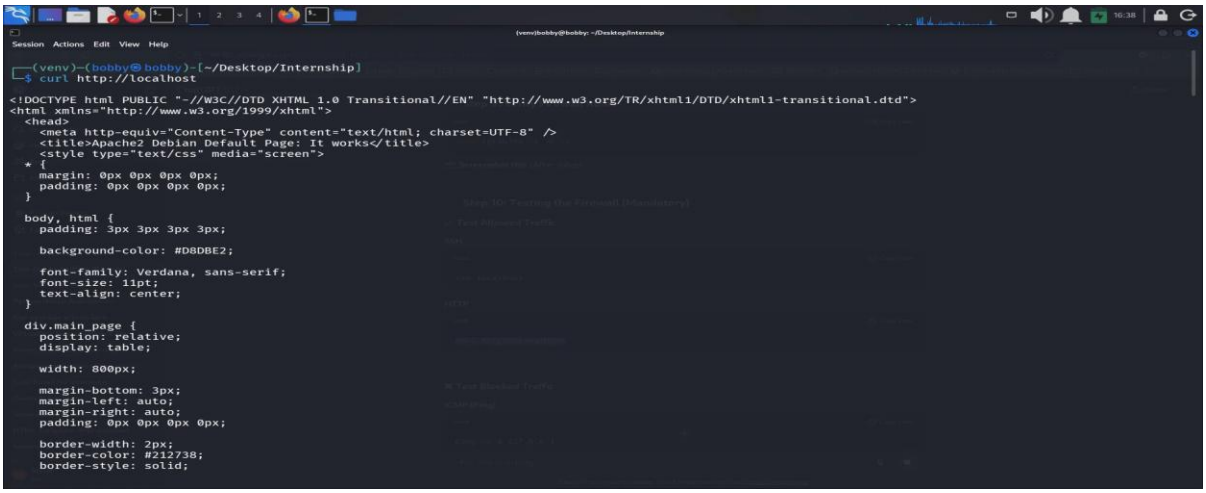
Testing the Firewall Configuration

Testing Allowed Traffic (HTTP)

HTTP traffic on port 80 was tested:

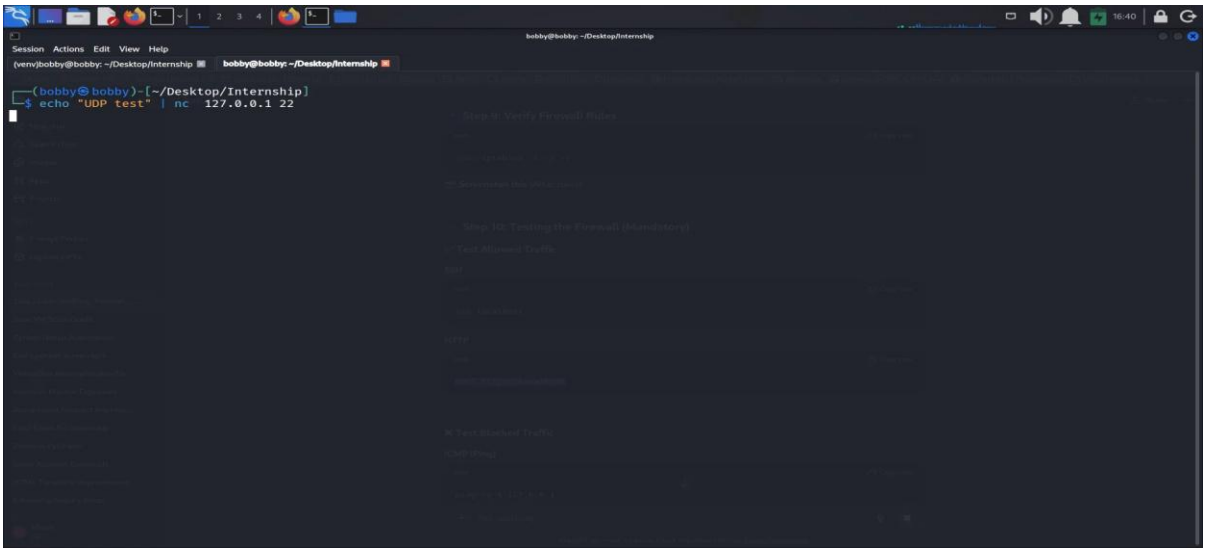
```
$ curl http://localhost
```

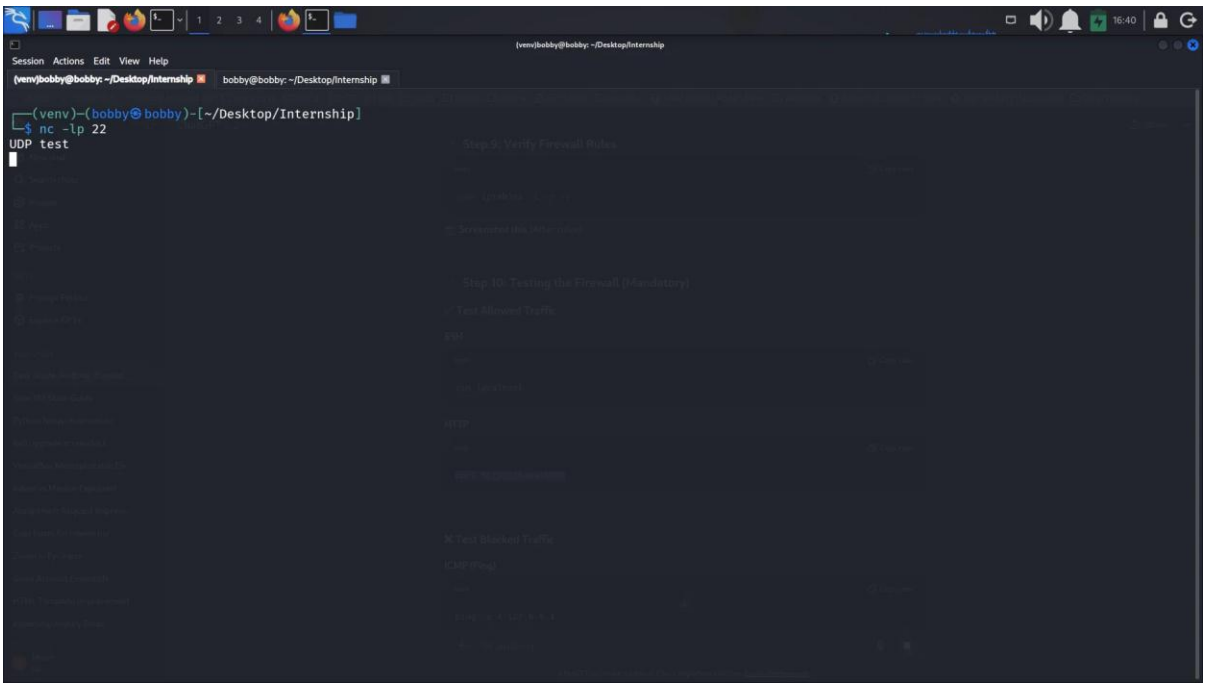
Result: Successfully retrieved Apache default page.



Testing Allowed Traffic (SSH)

SSH connectivity was tested using netcat:

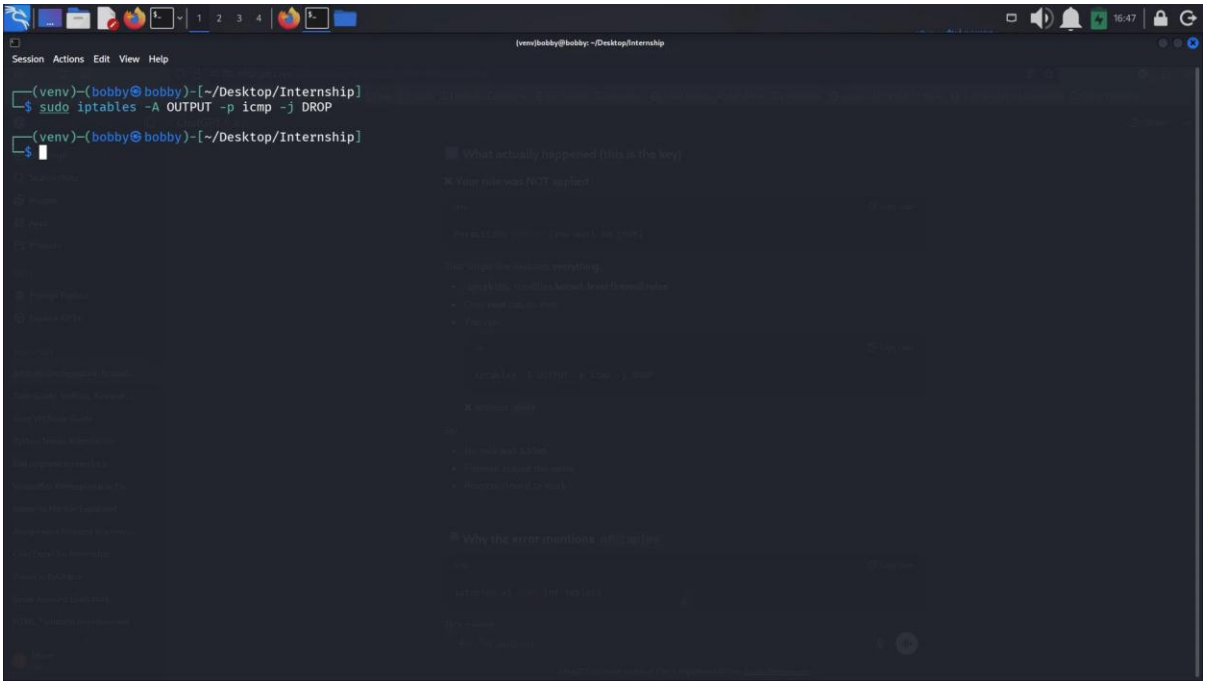




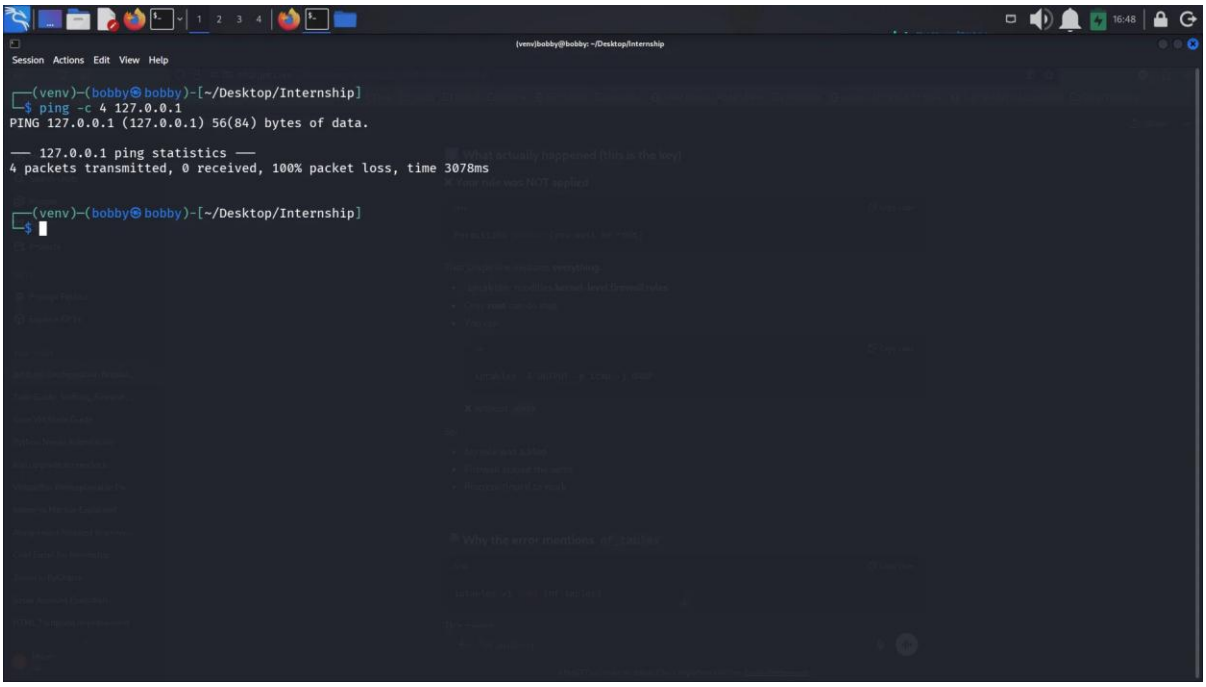
Testing Blocked Traffic (ICMP)

ICMP traffic was blocked to verify firewall effectiveness:

```
$ sudo iptables -A OUTPUT -p icmp -j DROP $ ping -c 4 127.0.0.1
```



Result: 100% packet loss, confirming successful blocking.



OpenVAS

What is OpenVAS?

OpenVAS (Open Vulnerability Assessment System) is a comprehensive open-source vulnerability scanner and management system. It is part of the Greenbone Vulnerability Management (GVM) framework and provides powerful capabilities for identifying security weaknesses in computer systems, networks, and applications.

Key Features

- **Comprehensive Vulnerability Database:** Contains over 50,000 Network Vulnerability Tests (NVTs) that are regularly updated to detect the latest security issues.
- **CVSS Scoring:** Uses the Common Vulnerability Scoring System (CVSS) to rate vulnerabilities from 0.0 to 10.0 based on severity.
- **Web-Based Interface:** Provides Greenbone Security Assistant (GSA) for easy scan management and report generation.
- **Automated Scanning:** Supports scheduled scans, continuous monitoring, and automated vulnerability detection.
- **Detailed Reporting:** Generates comprehensive reports with vulnerability details, remediation steps, and risk assessments.

OpenVAS Architecture

OpenVAS operates through multiple interconnected components:

- **GSA (Greenbone Security Assistant):** Web-based user interface for scan configuration and result viewing.
- **gvmd (Greenbone Vulnerability Manager Daemon):** Core management daemon that orchestrates scans and manages data.
- **OpenVAS Scanner:** The actual scanning engine that performs vulnerability tests on target systems.
- **NVT Feed:** Regularly updated database of Network Vulnerability Tests.
- **PostgreSQL Database:** Stores scan results, configurations, and vulnerability data.

Scanning Process

The OpenVAS scanning process follows these steps:

1. **Target Definition:** Define the IP addresses or hostnames to be scanned.
2. **Scan Configuration:** Select scan policy (Full and Fast, System Discovery, etc.) and configure scan parameters.
3. **Port Scanning:** Identify open ports and running services on target systems.
4. **Service Detection:** Determine versions and configurations of detected services.
5. **Vulnerability Testing:** Execute NVTs against identified services to detect known vulnerabilities.
6. **Results Analysis:** Compile findings, assign severity scores, and generate detailed reports.
7. **Remediation Guidance:** Provide recommendations and solutions for identified vulnerabilities.

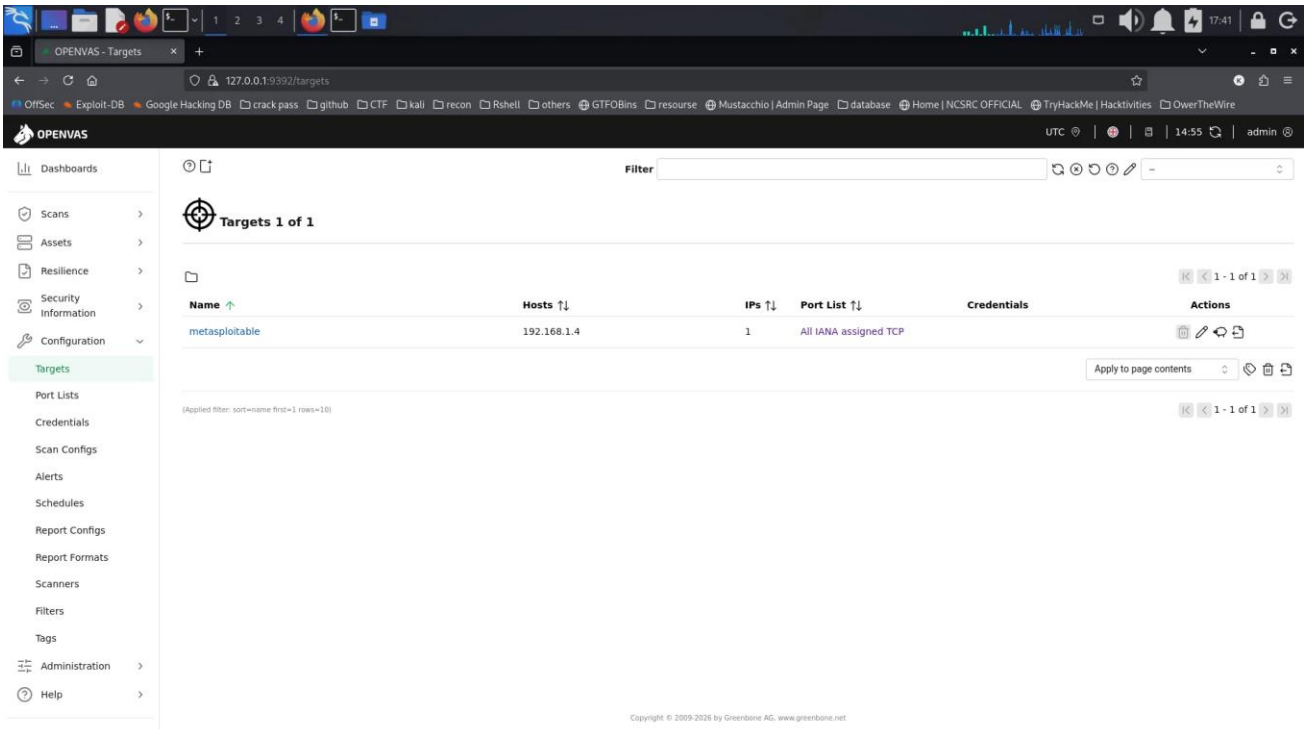
OPENVAS VULNERABILITY SCAN

Network Security Assessment Report
Target: Metasploitable VM
Scan Information
Target IP: 192.168.1.4
Scan Date: January 22, 2026
Scanner: OpenVAS 26.8.0 (GVM)
Task Name: meta
Scan Duration: 0:05 hours

OpenVAS Scan Implementation Process

Creating a Target

The first step in performing a vulnerability scan is to define the target system. In OpenVAS, navigate to Configuration → Targets to create a new target entry.



The task overview displays:

- **Task Name:** meta
- **Status:** New (shown in green, indicating the task is ready to run)
- **Severity Class:** N/A (no results yet as scan hasn't been run)
- **Dashboard Charts:** Show task distribution by severity and status

To initiate the scan, click the play/start button next to the task. The scan will then begin executing vulnerability tests against the target system.

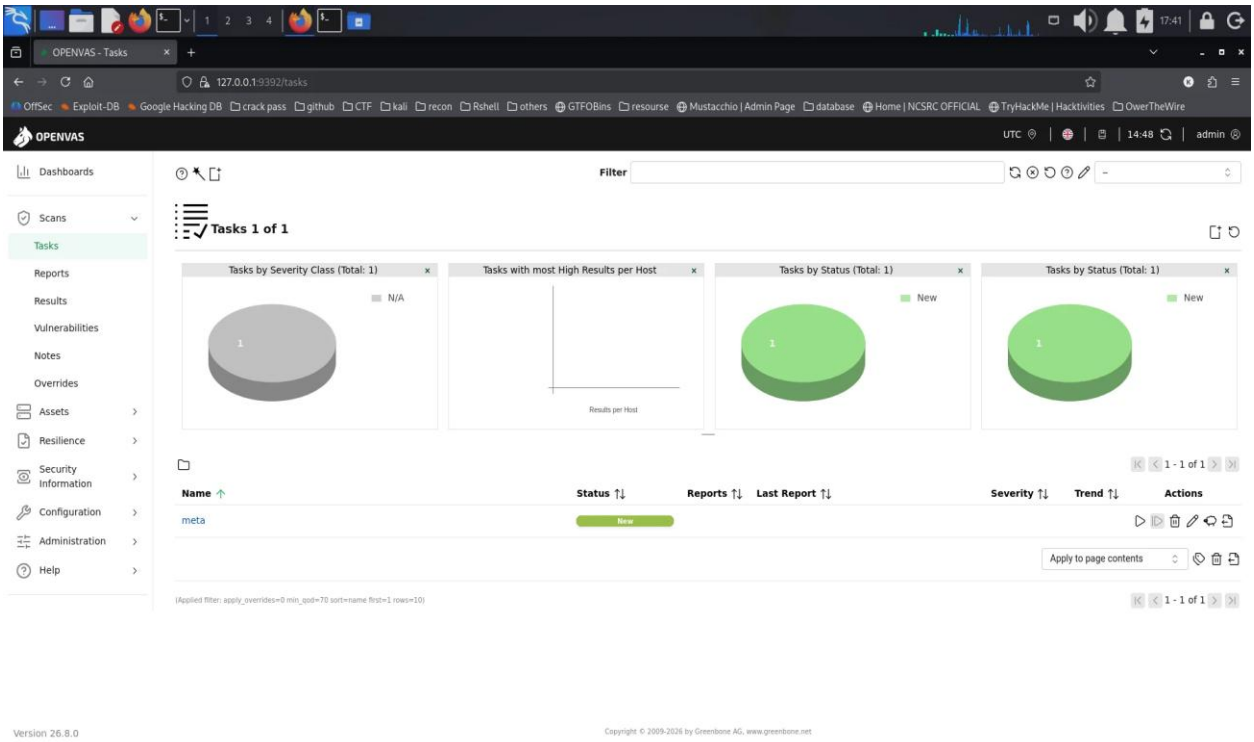
Creating and Running a Scan Task

After defining the target, a scan task must be created. Navigate to Scans → Tasks to create and configure a new scanning task.

As shown in the screenshot, the target configuration includes:

- **Name:** metasploitable (descriptive identifier for the target)
- **Hosts:** 192.168.1.4 (IP address of the Metasploitable VM)
- **Port List:** All IANA assigned TCP (scans all standard TCP ports)
- **Credentials:** None specified (unauthenticated scan)

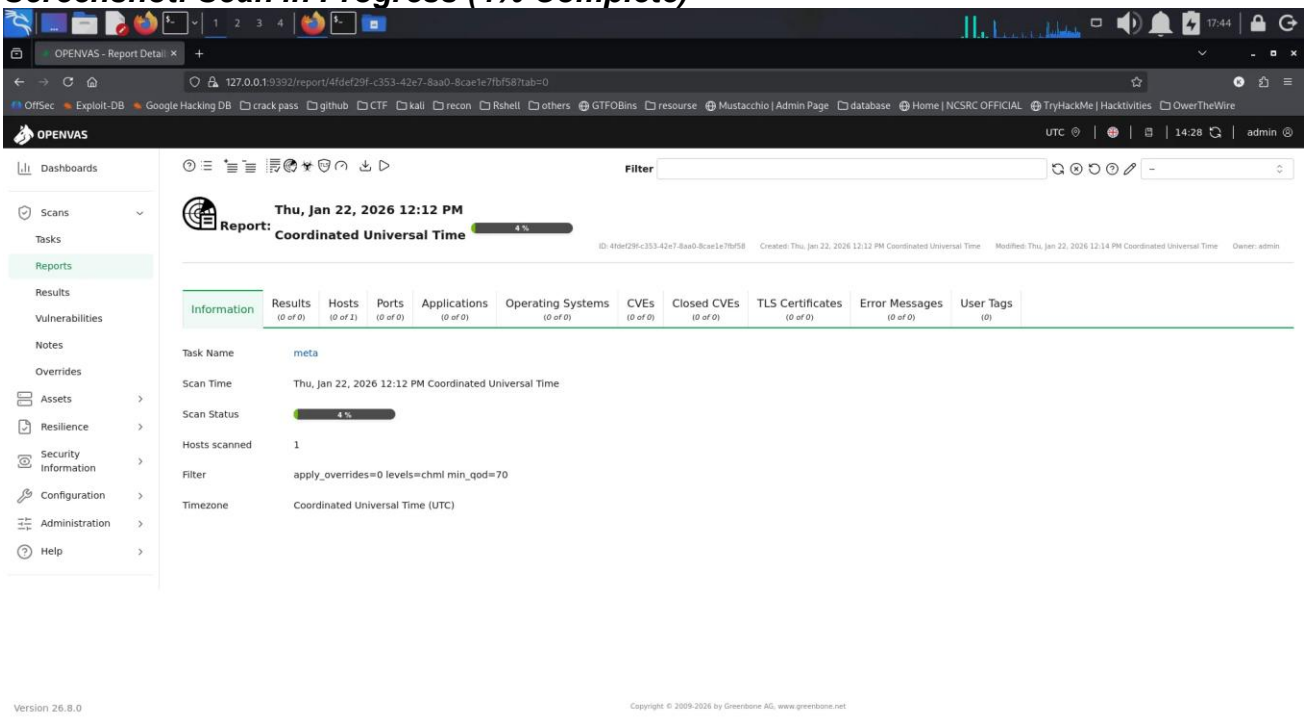
After that save task.



Monitoring Scan Progress

During the scan execution, OpenVAS provides real-time progress monitoring. Navigate to Scans → Reports to view the scan status and progress.

Screenshot: Scan in Progress (4% Complete)

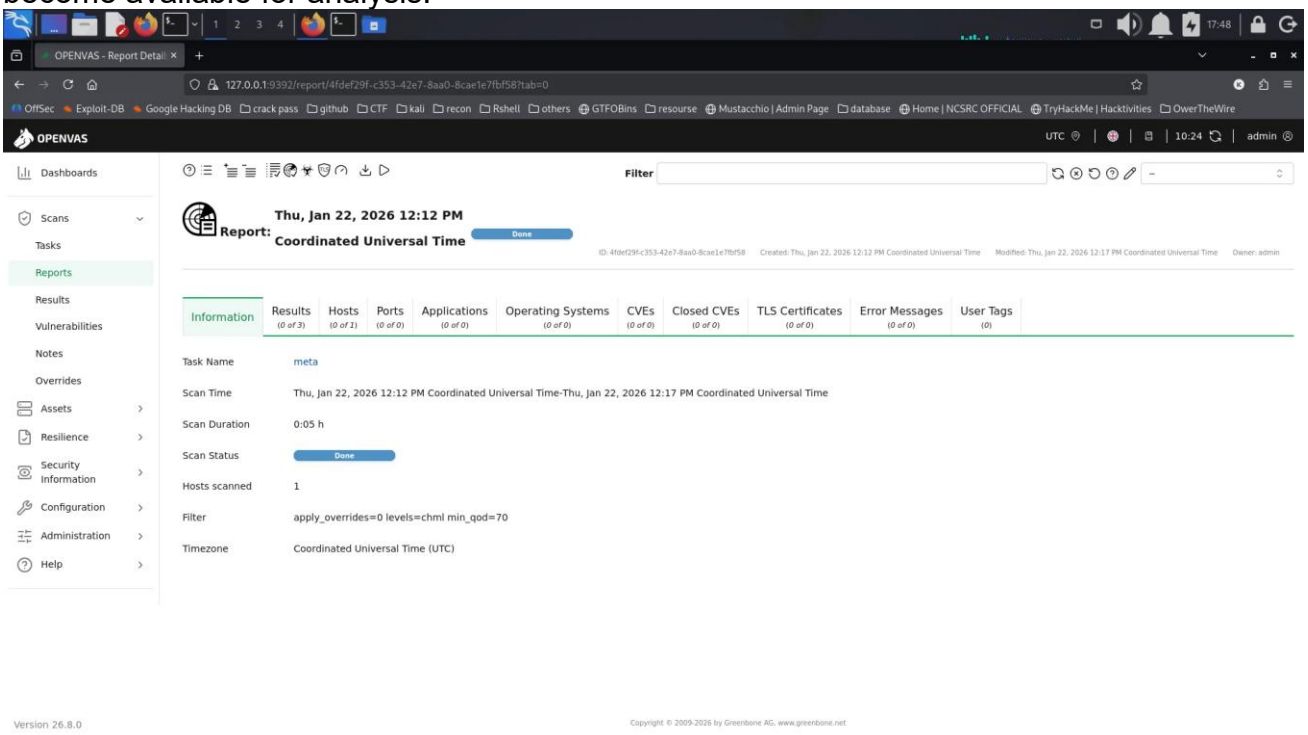


The progress screen shows the scan is 4% complete. Key information displayed:

- **Task Name:** meta
- **Scan Status:** 4% (progress bar showing active scanning)
- **Hosts Scanned:** 1
- **Scan Start Time:** Thu, Jan 22, 2026 12:12 PM Coordinated Universal Time
- **Applied Filter:** apply_overrides=0 levels=chml min_qod=70 (filtering parameters for results)

Scan Completion

Once the scan completes all vulnerability tests, the status changes to 'Done' and results become available for analysis.



The completed scan report shows:

- **Scan Status:** Done (shown in blue, indicating successful completion)
- **Scan Duration:** 0:05 hours (5 minutes total scan time)
- **Report ID:** 4fdcf29f-c353-42e7-8aa0-8cae1e7fbf581tab=0
- **Creation Time:** Thu, Jan 22, 2026 12:12 PM
- **Modification Time:** Thu, Jan 22, 2026 12:17 PM

Report Tabs Available:

- **Information:** General scan metadata and configuration details
- **Results:** Detailed vulnerability findings (0 of 3 shown initially)
- **Hosts:** Information about scanned hosts (0 of 1)
- **Ports:** Open ports discovered during scanning (0 of 0)
- **Applications, Operating Systems, CVEs:** Additional categorized findings